

Orbit



Índice

1	Visão Geral do Projeto	4
2	Stack Tecnológica	5
3	Arquitetura do Backend (Clean Architecture)	6
4	Entidades do Domínio	8
5	Sistema Multi-usuário / Famílias	10
6	Autenticação e Segurança	11
7	Funcionalidades Implementadas	12
8	API Reference — Endpoints	15
9	Frontend (Next.js)	17
10	Alertas Proativos e Lembretes	19
11	Integração com StarBot (Telegram)	20
12	Fluxo Completo: Usuário → Telegram → API	23
13	Docker e Deploy	24
14	Configuração de Ambiente	25



1. Visão Geral do Projeto

Orbit é uma plataforma completa de gestão financeira pessoal, desenvolvida com .NET 8 e Next.js. Cobre desde controle de transações do dia a dia até cálculo de DAS do MEI, simulação de dívidas e rastreamento de investimentos.



Propósito

Centralizar toda a vida financeira em um único lugar — contas, gastos, assinaturas, metas, dívidas e investimentos — com insights automáticos e integração ao Telegram.



Público-alvo

Uso pessoal e familiar. Suporte nativo a múltiplos usuários em uma família (FamilyId compartilhado), com rastreamento de quem criou cada registro.

Métricas do Projeto

60+

Endpoints na API

25+

Entidades do domínio

11

Ferramentas StarBot

15+

Páginas no frontend

6

Tipos de alertas proativos

4

Containers Docker



2. Stack Tecnológica

Backend

TECNOLOGIA	USO
.NET 8 / C# 12	Framework principal

Frontend

TECNOLOGIA	USO
Next.js 14	Framework React (App Router)

ASP.NET Core Web API	Controllers / Middleware
Entity Framework Core 8	ORM + Migrations
PostgreSQL 16	Banco de dados principal
Redis 7	Cache
MediatR 12	CQRS (Commands/Queries)
BCrypt.Net	Hash de senhas
Serilog	Logging estruturado
CsvHelper	Importação de CSV
FluentValidation	Validação de DTOs

TypeScript	Tipagem estática
Tailwind CSS	Estilização utilitária
TanStack Query v5	Data fetching / cache
Recharts	Gráficos e charts
Lucide React	Ícones

StarBot / DevOps

TECNOLOGIA	USO
Node.js 20 + TypeScript	Bot Telegram (StarBot)
grammy	Framework Telegram
@anthropic-ai/sdk	Claude AI no bot
Docker + Compose	Containerização
PM2	Process manager (VPS)



3. Arquitetura do Backend (Clean Architecture)

O backend segue a **Clean Architecture**, garantindo separação de responsabilidades e que a lógica de negócio não dependa de frameworks ou infraestrutura.

🌐 API LAYER — ORBIT.API

Controllers, Middleware, Program.cs · Recebe requisições HTTP, valida JWT, serializa respostas

AuthController TransactionsController BudgetsController InvestmentsController ReportsController
AlertsController ExceptionHandlingMiddleware BaseAuthenticatedController



⚡ APPLICATION LAYER — ORBIT.APPLICATION

CQRS (Commands + Queries), Handlers (MediatR), DTOs, Validators · Orquestra os casos de uso

CreateTransactionCommand ImportCsvCommand GetBudgetConsolidatedQuery
SimulateDebtPaymentCommand GetMeiDashboardQuery GetSubscriptionForecastQuery



❤️ DOMAIN LAYER — ORBIT.DOMAIN

Entidades, Enums, Interfaces, Exceções · Núcleo puro, sem nenhuma dependência externa

User Family Transaction Account Budget Goal Debt Investment Subscription Notification
Reminder IRepository<T> IAuthService



🔧 INFRASTRUCTURE LAYER — ORBIT.INFRASTRUCTURE

DbContext (EF Core), Repositories, Services, Migrations · Implementa as interfaces do Domain

ApplicationDbContext Repository<T> AuthService (JWT) TelegramService CurrencyService
AlertEvaluationService

Fluxo de uma Requisição HTTP

- 1 Cliente envia `POST /api/transactions` com JWT
Qualquer cliente: browser, app mobile ou StarBot
- 2 `ExceptionHandlingMiddleware` captura o request
Envolve toda a pipeline, captura qualquer exceção e retorna resposta padronizada
- 3 `Rate Limiter` verifica 100 req/min por usuário
Retorna 429 se ultrapassado

4

JWT Authentication valida o token

Extrai `UserId`, `FamilyId`, `Username`, `Role` das claims

5

TransactionsController.Create() cria o Command

Herda de `BaseAuthenticatedController` que expõe `UserId`, `FamilyId`, `Username`

6

MediatR roteia para CreateTransactionCommandHandler

Controller não conhece o handler diretamente (Mediator Pattern)

7

Handler valida, cria a entidade, salva via Repository

Busca `Account` e `Category`, valida `FamilyId`, cria `Transaction` com `CreatedByUsername`, atualiza saldo

8

Retorna TransactionDto com 201 Created

DTO inclui campos de auditoria: `CreatedAt`, `CreatedByUsername`, `UpdatedByUsername`



4. Entidades do Domínio

Todas as entidades herdam de `BaseEntity` (Id, CreatedAt, UpdatedAt) ou `AuditableEntity` (adiciona CreatedByUsername, UpdatedByUsername). Todos os dados são isolados por **FamilyId**.

Hierarquia base

```
public class BaseEntity
{
    public Guid Id { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
}

public class AuditableEntity : BaseEntity
{
    public string? CreatedByUsername { get; set; }
    public string? UpdatedByUsername { get; set; }
}
```

Principais Entidades

ENTIDADE	HERDA DE	DESCRIÇÃO	RELACIONAMENTOS
Family	BaseEntity	Grupo familiar, unidade de isolamento	→ Users, Reminders
User	BaseEntity	Usuário com email, senha (BCrypt), role	→ Family
Account	AuditableEntity	Conta bancária com saldo e moeda	FamilyId, AccountType, Currency
Category	BaseEntity	Categoria de transação com cor e ícone	FamilyId
Transaction	AuditableEntity	Receita ou despesa com tags	→ Account, → Category, FamilyId
Budget	AuditableEntity	Orçamento mensal por categoria	→ Category, FamilyId

Subscription	AuditableEntity	Assinaturas recorrentes (Netflix, etc)	→ Account, → Category, FamilyId
Goal	AuditableEntity	Meta financeira com progresso	FamilyId, GoalStatus
Debt	AuditableEntity	Dívida com juros, estratégia de pagamento	FamilyId, DebtPaymentStrategy
Investment	AuditableEntity	Ativo (ação, FII, tesouro) com preço atual	FamilyId, InvestmentType, → Account?
InvestmentTransaction	BaseEntity	Compra/venda de ativo	→ Investment
Notification	BaseEntity	Notificação in-app com link	FamilyId, NotificationType
Reminder	AuditableEntity	Lembrete por data (aniversário, etc)	→ Family, DaysInAdvance
TransactionTemplate	BaseEntity	Template reutilizável de transação	FamilyId, → Account, → Category
CurrencyRate	BaseEntity	Taxa de câmbio (open.er-api.com)	FromCurrency, ToCurrency, Rate
MeiSettings	AuditableEntity	Configurações MEI (limite anual, CNAE)	FamilyId
WorkCalendarSettings	AuditableEntity	Config do calendário PJ (valor hora/dia)	FamilyId
WorkDay	BaseEntity	Dia trabalhado com horas e observação	→ WorkCalendarSettings
Holiday	BaseEntity	Feriado / dia não trabalhado	→ WorkCalendarSettings
ShoppingList	AuditableEntity	Lista de compras com status	FamilyId, ShoppingListStatus
ShoppingItem	BaseEntity	Item da lista com prioridade e preço	→ ShoppingList, ItemPriority
ClassificationRule	BaseEntity	Regra de classificação automática	FamilyId, Keyword → CategoryId
RoundupRule	AuditableEntity	Regra de microinvestimento por arredondamento	FamilyId, → Account



5. Sistema Multi-usuário / Famílias

O Orbit suporta múltiplos usuários compartilhando os mesmos dados financeiros através do conceito de **Family**. Todos os dados são isolados por `FamilyId`.

Estrutura

- `Family` é a unidade de isolamento
- Cada `User` pertence a uma `Family`
- Todos os recursos têm `FamilyId`
- Queries sempre filtram por FamilyId do token JWT

Roles

- `Admin` — acessa painel administrativo, pode gerenciar usuários da família
- `Member` — acessa todos os dados da família, sem admin
- Verificado via `[AdminOnly]` attribute nos controllers

Como FamilyId é propagado

```
// BaseAuthenticatedController.cs
public abstract class BaseAuthenticatedController : ControllerBase
{
    protected Guid UserId => Guid.Parse(
        User.FindFirstValue(ClaimTypes.NameIdentifier!));

    protected Guid FamilyId => Guid.Parse(
        User.FindFirstValue("family_id!));

    protected string Username =>
        User.FindFirstValue(ClaimTypes.Name!);

    protected bool IsAdmin =>
        User.IsInRole("Admin");
}
```

Auditoria de campos

Ao criar ou atualizar qualquer recurso, o username é gravado:

```
// Em todos os controllers que herdam de BaseAuthenticatedController:
// Criar:
entity.CreatedByUsername = Username;
```

```
// Atualizar:  
entity.UpdatedByUsername = Username;
```

O frontend exibe isso como "Criado por @tiago" na lista de transações.



6. Autenticação e Segurança

JWT com Refresh Token

1

Login: POST /api/auth/login

Valida email + senha (BCrypt). Retorna `accessToken` (15min) + `refreshToken` (7 dias)

2

Requests autenticados

Header: `Authorization: Bearer <accessToken>`. Claims: UserId, FamilyId, Username, Role

3

Refresh: POST /api/auth/refresh

Troca o refreshToken por um novo par access+refresh. O frontend faz isso automaticamente via interceptor em `coreApi.ts`

Proteções implementadas

Rate Limiting

100 req/min por usuário. Retorna 429 se ultrapassado.

Resource Ownership

Cada query filtra por FamilyId — impossível acessar dados de outra família.

CORS

Configurado para permitir apenas origens definidas em appsettings.

BCrypt

Senhas nunca armazenadas em plain text. Work factor configurável.



7. Funcionalidades Implementadas



Contas Bancárias

Corrente, Poupança, Cartão, Investimento, Carteira, Negócio. Suporte a multi-moeda (BRL, USD, EUR). Saldo atualizado automaticamente a cada transação.



Transações

CRUD completo com suporte a tags, descrição e vínculo com conta e categoria. Importação via CSV. Classificação automática (aprende com edições). Paginação (25/página). Detecção de duplicatas.



Orçamento Mensal

Por categoria, com percentual de uso em tempo real. Alertas automáticos ao atingir 80%. Visão consolidada com gasto vs limite.



Assinaturas

Gerenciamento completo com status (Ativa/Pausada/Cancelada). Alertas 3 dias antes do vencimento. Detecção de assinaturas sub-utilizadas. Forecast 30 dias. Geração de transações recorrentes.



Metas Financeiras

Objetivos com prazo e valor-alvo. Contribuições incrementais. Barra de progresso. Status automático (Active/Achieved/Cancelled).



Gestão de Dívidas

Registro com juros e valor total (opcional, para dívidas indeterminadas). Simulação por estratégia Snowball (menor dívida primeiro) ou Avalanche (maior juros primeiro). Comparativo de economia entre estratégias.



Investimentos

Portfolio com ações, FIIs, Tesouro Direto. Rastreamento de preço médio vs atual. P&L calculado. Histórico de compras/vendas. Gráfico de alocação por tipo.



Relatórios

Relatório mensal com receitas/despesas por categoria. Previsão de fluxo de caixa (30/60/90 dias). Comparação entre períodos.



MEI — Calculadora DAS

Painel com faturamento anual acumulado vs limite (R\$ 81k). Estimativa do DAS mensal com breakdown (INSS, ICMS/ISS). Gráfico de consumo do limite anual.



Calendário de Trabalho (PJ)

Registro de dias trabalhados, horas por dia e valor hora/dia. Controle de feriados. Resumo mensal com total de horas e receita projetada.



Notificações In-App



Multi-Moeda

Bell icon na navbar com badge de não-lidas.
Dropdown com lista. Tipos: Info, Warning, Error, Success. Geradas por alertas proativos e sistema de lembretes.

Contas com moeda diferente (BRL, USD, EUR, etc).
Taxa de câmbio atualizada via open.er-api.com.
Endpoint de conversão. Refresh manual das taxas.



Templates de Transação

Salvar uma transação como template para reutilização rápida. Aplicar template cria a transação instantaneamente. Ideal para gastos recorrentes que não são assinaturas.



Listas de Compras

Listas com status (Pendente/Em Progresso/Concluída). Itens com prioridade (Baixa/Média/Alta/Urgente) e preço estimado. Totalizador automático.



Classificação Automática

Sistema ML simples que aprende com edições. Ao categorizar "Uber" como Transporte, cria regra keyword → categoria. Na próxima importação, sugere automaticamente.



Microinvestimentos (Roundup)

Arredonda transações e investe a diferença. Regras configuráveis com multiplicador (1x, 2x, etc). Simulação mensal. Transferência automática entre contas.



8. API Reference — Endpoints

Auth

POST /api/auth/signup

POST /api/auth/login

POST /api/auth/refresh

Usuário

GET /api/user/me

Contas

GET /api/accounts

POST /api/accounts

PUT /api/accounts/{id}

DEL /api/accounts/{id}

Transações

GET /api/transactions

POST /api/transactions

POST /api/transactions/import/csv

GET /api/transactions/summary

GET /api/transactions/duplicates

PUT /api/transactions/{id}

DEL /api/transactions/{id}

Categorias

Assinaturas

GET /api/subscriptions

POST /api/subscriptions

POST /api/subscriptions/forecast

POST /api/subscriptions/generate-billings

PUT /api/subscriptions/{id}

DEL /api/subscriptions/{id}

Dívidas

GET /api/debts

POST /api/debts

POST /api/debts/simulate

PUT /api/debts/{id}

DEL /api/debts/{id}

Investimentos

GET /api/investments

GET /api/investments/summary

POST /api/investments

POST /api/investments/{id}/transactions

PUT /api/investments/{id}

DEL /api/investments/{id}

Relatórios

GET /api/categories

POST /api/categories

PUT /api/categories/{id}

DEL /api/categories/{id}

Orçamentos

GET /api/budgets

GET /api/budgets/consolidated

POST /api/budgets

PUT /api/budgets/{id}

DEL /api/budgets/{id}

Metas

GET /api/goals

POST /api/goals

POST /api/goals/{id}/contribute

PUT /api/goals/{id}

DEL /api/goals/{id}

GET /api/reports/monthly

GET /api/reports/cashflow

GET /api/reports/comparison

Notificações

GET /api/notifications

GET /api/notifications/unread-count

POST /api/notifications/{id}/read

POST /api/notifications/read-all

Alertas

GET /api/alert-configurations

POST /api/alert-configurations

PUT /api/alert-configurations/{id}

DEL /api/alert-configurations/{id}

Lembretes

GET /api/reminders

POST /api/reminders

PUT /api/reminders/{id}

POST /api/reminders/{id}/toggle

DEL /api/reminders/{id}

Câmbio

GET /api/currency/rates

GET /api/currency/convert

POST /api/currency/refresh



9. Frontend (Next.js)

O frontend usa Next.js 14 com App Router, TypeScript e Tailwind CSS. Toda comunicação com a API passa pelos módulos em `src/lib/api/`.

Páginas

ROTA	COMPONENTE PRINCIPAL	DESCRIÇÃO
<code>/login</code>	LoginPage	Autenticação JWT
<code>/dashboard</code>	DashboardPage	Resumo financeiro + contas + saldo + próximas contas
<code>/transactions</code>	TransactionsPage	Lista paginada + filtros + import CSV + templates
<code>/budgets</code>	BudgetsPage	Orçamentos por categoria + progresso
<code>/monthly-bills</code>	MonthlyBillsPage	Assinaturas + contas fixas mensais
<code>/subscriptions</code>	SubscriptionsPage	Gestão de assinaturas recorrentes
<code>/goals</code>	GoalsPage	Metas com progresso e contribuições
<code>/debts</code>	DebtsPage	Dívidas + simulador Snowball/Avalanche
<code>/investments</code>	InvestmentsPage	Portfolio + allocation chart + P&L
<code>/reports</code>	ReportsPage	Relatório mensal + cashflow + comparação
<code>/statement</code>	StatementPage	Extrato completo de todas as contas
<code>/shopping-lists</code>	ShoppingListsPage	Listas de compras com itens e prioridades
<code>/alerts</code>	AlertsPage	Configuração de alertas + lembretes (2 tabs)
<code>/trabalho</code>	TrabalhoPage	Calendário PJ + calculadora MEI DAS
<code>/settings</code>	SettingsPage	Perfil, contas, regras de classificação
<code>/admin</code>	AdminPage	Gestão de usuários da família (Admin only)

Layout e Navegação

Sidebar (desktop)

- Largura: 260px (expandida) / 72px (colapsada)
- Ícone + label nas opções
- Collapsible via botão toggle
- Mostra @username e nome da família

MobileNav (mobile)

- Bottom bar com ícones das principais páginas
- Drawer lateral para itens secundários
- Hamburger menu para abrir o drawer

Padrão de API no Frontend

```
// src/lib/api/transactionsApi.ts (padrão de todos os módulos)
export const transactionsApi = {
  getAll: async (params?: GetTransactionsParams) => {
    const res = await coreApi.get('/transactions', { params });
    return res.data as Transaction[];
  },
  create: async (data: CreateTransactionRequest) => {
    const res = await coreApi.post('/transactions', data);
    return res.data as Transaction;
  },
  // ...update, delete, importCsv, getSummary
};
```

`coreApi.ts` é um wrapper do Axios com interceptors para injetar o JWT e fazer refresh automático do token quando recebe 401.

Design System

Paleta de Cores

- **Emerald/Teal** — cor principal da marca (gradient `from-emerald-600 to-teal-600`)
- **Blue** — apenas variantes "info" (Badge, Alert, Notification)
- Modo claro e escuro (dark mode nativo via Tailwind)

Componentes UI

- `Button`, `Input`, `Select`
- `Modal`, `Tabs`
- `LoadingSpinner`
- `PageContainer` — wraps o conteúdo de cada página



10. Alertas Proativos e Lembretes

O `AlertEvaluationService` é um **BackgroundService** do .NET que roda a cada N horas (configurável via env var `ALERT_EVAL_INTERVAL`, padrão 6h). Ele avalia todas as famílias e dispara alertas quando necessário.

Tipos de Alerta Avaliados

TIPO	CONDIÇÃO DE DISPARO	DESTINO
<code>BudgetWarning</code>	Orçamento de alguma categoria $\geq$ 80% do limite mensal	InApp
<code>BillDue</code>	Assinatura com vencimento nos próximos 3 dias	InApp
<code>DebtDue</code>	Dívida com próximo pagamento em $\leq$ 7 dias	InApp
<code>NegativeBalance</code>	Alguma conta com saldo negativo	InApp + Telegram
<code>GoalNearTarget</code>	Meta financeira atingiu $\geq$ 90% do valor alvo	InApp
<code>LastBusinessDay</code>	Último dia útil do mês (para fechamento MEI, etc)	InApp + Telegram
<code>Reminder</code>	Lembrete configurado com <code>DaysInAdvance</code> antes da data	InApp + Telegram

Deduplicação: O service verifica se já existe uma Notification com o mesmo título criada hoje para a família antes de criar nova. Isso evita spam em execuções repetidas.

TelegramService

Envia mensagens direto para o usuário via **Telegram Bot API**:

```
// Infrastructure/Services/TelegramService.cs
public async Task SendMessageAsync(string message)
{
    // POST https://api.telegram.org/bot{TOKEN}/sendMessage
    // { chat_id: CHAT_ID, text: message, parse_mode: "HTML" }
}
```

Configuração via env vars: `TELEGRAM_BOT_TOKEN` e `TELEGRAM_CHAT_ID`.

Lembretes

A entidade `Reminder` permite configurar lembretes por data (mês/dia):

- Exemplos: aniversários, renovação de contratos, vencimento de apólice
- `IsRecurring`: se é anual (ex: aniversário) ou único
- `DaysInAdvance`: quantos dias antes avisar (padrão: 3)
- O `AlertEvaluationService` envia notificação InApp + Telegram ao atingir a janela



11. Integração com StarBot (Telegram)

O **StarBot** é um bot do Telegram com IA (Claude da Anthropic) que se integra ao Orbit. O usuário pode conversar naturalmente em português e o bot usa ferramentas (tools) para consultar e registrar dados na API.

→ Telegram

Interface de conversa. Usuário digita em linguagem natural: *"quanto gastei este mês?"*, *"registra um gasto de 50 reais no almoço"*.

🗨️ Claude AI

Interpreta a mensagem, decide qual ferramenta usar (ou combinar múltiplas), executa, e formula uma resposta em português natural e formatada para Telegram.

Arquitetura do StarBot

```
starbot/src/
├── ai/providers/
│   └── AnthropicProvider.ts    ← passa tools p/ API + parseia tool_use
├── channels/telegram/
│   └── TelegramAdapter.ts     ← loop de execução de ferramentas (max 10 iter)
├── routing/
│   └── MessageRouter.ts       ← orquestra providers + ferramentas
├── tools/
│   ├── index.ts               ← ToolRegistry + registro condicional
│   └── builtin/finance/
│       ├── index.ts           ← getAllFinanceTools()
│       ├── ListAccountsTool.ts
│       ├── ListTransactionsTool.ts
│       ├── CreateTransactionTool.ts
│       ├── ListCategoriesTool.ts
│       ├── MonthlyReportTool.ts
│       ├── SubscriptionsTool.ts
│       ├── GoalsTool.ts
│       ├── DebtsTool.ts
│       └── BudgetsTool.ts
└── services/
    └── FinanceApiClient.ts    ← HTTP client com JWT auth + auto-refresh
```

FinanceApiClient

Cliente HTTP em TypeScript que gerencia autenticação automaticamente:

```
// services/FinanceApiClient.ts
export class FinanceApiClient {
  private accessToken: string | null = null;
  private refreshToken: string | null = null;

  // Login automático + cache do token
  async ensureAuthenticated(): Promise<void> { ... }

  // Refresh automático em 401
  async request<T>(method, path, data?): Promise<T> {
    if (response.status === 401) {
      await this.refreshAccessToken();
      return this.request(method, path, data); // retry
    }
  }
}
```

Ferramentas Disponíveis (11 tools)

list_accounts

Lista todas as contas com saldo atual, tipo e moeda. O bot usa isso para mostrar o patrimônio total ou responder "qual meu saldo?".

list_categories

Lista categorias disponíveis com Id, nome e tipo (Income/Expense). Necessário para criar transações corretamente.

create_transaction

Cria uma receita ou despesa. Parâmetros: accountId, categoryId, amount, type, description, date, tags. O bot primeiro lista contas e categorias se não souber os IDs.

list_transactions

Lista transações com filtros: startDate, endDate, type, accountId, limit. Usado para "o que gastei hoje?" ou "últimas 10 transações".

monthly_report

Resumo do mês: total de receitas, despesas, saldo, transações por categoria. Parâmetros: month, year.

list_subscriptions

Lista assinaturas ativas com valor mensal e próximo vencimento. Útil para "quanto gasto em assinaturas?".

list_goals

Lista metas financeiras com progresso (currentAmount vs targetAmount). Mostra percentual e status.

list_debts

Lista dívidas com saldo devedor, juros mensais e data de vencimento. Calcula total de dívidas.

list_budgets

Lista orçamentos mensais com gasto vs limite e percentual usado. Útil para "quanto ainda posso gastar em alimentação?".

System Prompt do Bot

"Você é o assistente financeiro do Orbit. Você tem acesso às ferramentas de finanças do usuário. Ajude com consultas de saldo, registro de gastos e receitas, análise de orçamento, acompanhamento de metas e muito mais. Responda sempre em português brasileiro, de forma clara e formatada para Telegram. Use emojis moderadamente para melhorar a legibilidade."

Configuração (env vars do StarBot)

VARIÁVEL	DESCRIÇÃO	EXEMPLO
TELEGRAM_BOT_TOKEN	Token do bot (@BotFather)	123456:ABCdef...
ANTHROPIC_API_KEY	Chave da API do Claude	sk-ant-...
FINANCE_API_URL	URL base da API Orbit	http://localhost:5000
FINANCE_USERNAME	Email do usuário Orbit	tiago@email.com
FINANCE_PASSWORD	Senha do usuário Orbit	Senha123!



12. Fluxo Completo: Usuário → Telegram → API

Este é o fluxo end-to-end de como o usuário interage com o Orbit via Telegram:

- 1 Usuário envia mensagem no Telegram**
Exemplo: "registra um gasto de 45 reais no iFood agora"
- 2 grammy recebe o update via Telegram Bot API**
`TelegramAdapter` processa a mensagem e inicia o loop de ferramentas
- 3 MessageRouter envia para AnthropicProvider (Claude)**
A conversa é enviada para a API do Claude junto com as 11 definições de tools
- 4 Claude decide: precisa das contas e categorias primeiro**
Retorna `tool_use: list_accounts` e `tool_use: list_categories`
- 5 StarBot executa as ferramentas contra a API Orbit**
`FinanceApiClient.get('/accounts')` → JWT auth → PostgreSQL → retorna lista de contas e categorias
- 6 Claude recebe os resultados e decide criar a transação**
Identifica conta principal, categoria "Alimentação", retorna `tool_use: create_transaction` com os dados completos
- 7 StarBot chama POST /api/transactions**
API valida JWT → verifica FamilyId → cria Transaction → atualiza saldo da conta → retorna TransactionDto
- 8 Claude formula a resposta final**
Com todos os tool_results em mãos, gera resposta em português natural formatada para Telegram
- 9 Usuário recebe confirmação no Telegram**
Exemplo: "✅ Gasto registrado!\n💰 R\$ 45,00 em Alimentação\n🍽️ iFood\n🏠 Conta: Carteira (saldo: R\$ 1.230,00)"

Loop de ferramentas: O TelegramAdapter e MessageRouter implementam um loop (máx 10 iterações) que continua enviando tool_results de volta para o Claude até que ele retorne uma resposta de texto final (sem mais tool_use). Isso permite encadear múltiplas ferramentas em uma só interação.

Exemplos de Conversas

Consulta de saldo

Usuário: "qual meu saldo total?"

Claude usa: list_accounts

Bot: "💰 Saldo Total: R\$ 3.450,00

- 💳 Nubank: R\$ 1.200,00
- 🏠 Bradesco: R\$ 2.150,00
- 🍎 Carteira: R\$ 100,00"

Relatório mensal

Usuário: "como ficou fevereiro?"

Claude usa: monthly_report(month=2, year=2026)

Bot: "📊 Fevereiro/2026

- ♥️ Receitas: R\$ 8.000
- 🔴 Despesas: R\$ 5.432
- 📈 Saldo: +R\$ 2.568"



13. Docker e Deploy

Containers

SERVIÇO	IMAGEM	PORTA	FUNÇÃO
api	Multi-stage .NET 8	5000	Backend API + Swagger
frontend	Multi-stage Node 20	3000	Next.js App
postgres	postgres:16-alpine	5432	Banco de dados
redis	redis:7-alpine	6379	Cache

Segurança: Nunca exponha as portas 5432 (PostgreSQL) e 6379 (Redis) para a internet. Apenas 22 (SSH), 3000 (Frontend) e 5000 (API) devem ser abertas no firewall da VPS.

Comandos

```
# Subir todos os containers
docker compose up -d --build

# Ver logs
docker compose logs -f api

# Backup do banco
docker exec orbit-postgres pg_dump -U postgres orbit > backup.sql

# Restaurar
cat backup.sql | docker exec -i orbit-postgres psql -U postgres orbit
```

Deploy na Oracle Cloud (Free Tier)

1

Criar VM na Oracle Cloud

Shape: VM.Standard.A1.Flex (ARM) — 2 OCPU + 6GB RAM, totalmente gratuito. Ubuntu 22.04.

2

Abrir portas no Security List

Ingress rules: TCP 22 (SSH), TCP 3000 (frontend), TCP 5000 (API)

3

Instalar Docker + clonar repositório

```
curl -fsSL https://get.docker.com | sudo sh + git clone <repo>
```

- 4

Configurar .env com IP público

Atualizar `NEXT_PUBLIC_API_URL=http://<IP>:5000/api` no docker-compose.yml
- 5

docker compose up -d --build

Primeira build leva 5-10min. Migrations aplicadas automaticamente ao iniciar.
- 6

StarBot com PM2 na mesma VPS (opcional)

`pm2 start dist/cli/index.js --name starbot -- start`. FINANCE_API_URL aponta para localhost:5000.



14. Configuração de Ambiente

Variáveis do Backend (.env)

VARIÁVEL	DESCRIÇÃO	PADRÃO/EXEMPLO
<code>DB_USER</code>	Usuário PostgreSQL	<code>postgres</code>
<code>DB_PASSWORD</code>	Senha PostgreSQL	<code>SenhaSeg@123</code>
<code>DB_NAME</code>	Nome do banco	<code>orbit</code>
<code>JWT_SECRET_KEY</code>	Chave para assinar tokens JWT (≥32 chars)	<i>chave única e secreta</i>
<code>SEED_ADMIN_PASSWORD</code>	Senha do admin criado no seed inicial	<code>Admin@123</code>
<code>TELEGRAM_BOT_TOKEN</code>	Token do bot para alertas proativos	<code>123:ABCdef...</code>
<code>TELEGRAM_CHAT_ID</code>	Chat ID para receber alertas	<code>123456789</code>
<code>ALERT_EVAL_INTERVAL</code>	Intervalo de avaliação de alertas (horas)	<code>6</code>

Variáveis do Frontend

VARIÁVEL	DESCRIÇÃO	EXEMPLO
<code>NEXT_PUBLIC_API_URL</code>	URL base da API (acessível pelo browser)	<code>http://localhost:5000/api</code>

Variáveis do StarBot

VARIÁVEL	DESCRIÇÃO	EXEMPLO
<code>TELEGRAM_BOT_TOKEN</code>	Token do bot Telegram	<code>123:ABCdef...</code>
<code>ANTHROPIC_API_KEY</code>	Chave da API Claude (Anthropic)	<code>sk-ant-...</code>

FINANCE_API_URL

URL da API Orbit

http://localhost:5000

FINANCE_USERNAME

Email de login no Orbit

user@email.com

FINANCE_PASSWORD

Senha de login no Orbit

Senha@123



Orbit

Plataforma completa de gestão financeira com IA via Telegram

.NET 8 · Next.js 14 · PostgreSQL · Claude AI · Docker · Oracle Cloud